

Soluzione Cross-Platform per ARM-GDK

Università degli Studi di Roma Tor Vergata

Anno accademico 2025/2026

Facoltà di Informatica

Prof. A. Simonetta, Ing. E. Iannaccone

Mohamed Abd Rabbou, Aurora Marini, Tiziana Monacelli

Sommario:

- Introduzione
- Requisiti di sistema
- Difficoltà incontrate
- Primo approccio risolutivo (tramite WSL)
- Soluzione finale (tramite Docker)
- Possibilità di una “Dual-Mode”
- Analisi e studio di fattibilità
- Miglioramenti possibili

Introduzione:

Il progetto nasce con l'obiettivo di creare una **soluzione Cross-Platform** che supporti l'esecuzione su **Windows** del **codice Assembly ARM**, inizialmente concepito unicamente per Linux. L'intento era quello di evitare l'utilizzo di emulatori, puntando ad una ricompilazione che permettesse al codice del programma di interagire direttamente con l'hardware, senza livelli di astrazione intermedi e di avviarsi rapidamente richiedendo un solo click.

Tabella degli obiettivi prefissati:

<u>Obiettivi chiave</u>	<u>Stato</u>
Accessibilità da Windows	Raggiunto
Avvio “one-click”	Raggiunto
Porting nativo	Non raggiunto (è stato necessario l'uso di un emulatore)

<u>Obiettivi aggiuntivi</u>	<u>Stato</u>
Portabilità multiplatforma (oltre a Windows)	Raggiunto
Compilazione automatica	Raggiunto
Visualizzazione dei flag	Non raggiunto (rimandato agli sviluppi futuri per limitazione dell'interfaccia grafica)
Visualizzazione dell'output	Parzialmente raggiunto (questa funzione è presente solo in una versione del progetto)

Requisiti di sistema:

Per poter utilizzare il progetto bisogna soddisfare i seguenti requisiti:

- Sistema operativo: Windows 10/ 11
- Software necessario: Docker Desktop
- IDE: VS Code

Difficoltà incontrate:

Durante la fase iniziale di sviluppo, abbiamo riscontrato degli ostacoli importanti nel tentativo di eseguire il programma in modo nativo tramite **Windows PowerShell**. La causa principale di questo imprevisto è la natura stessa del software: il codice core `main.s` è scritto in **Assembly ARM nativo** e la toolchain genera esclusivamente file binari in formato **ELF**, standard tipico dei sistemi UNIX-like, del tutto **incompatibile** con l'architettura x86_64 e lo standard **PE** (Portable Executable) di Windows.

L'opzione di riscrittura completa del codice per x86_64 è stata scartata per via della sua complessità e, soprattutto, dei limitati tempi a disposizione per il progetto. Il team ha quindi deciso di far fronte al problema orientandosi verso l'**emulazione** e la **virtualizzazione**.

Primo approccio risolutivo (tramite WSL):

Analizzato il problema di incompatibilità, la soluzione scelta è stata quella di adottare **WSL (Windows Subsystem for Linux)** e di emulare il programma tramite **QEMU (Quick Emulator)**, accoppiato al **debugger GDB** e all'interfaccia grafica web **GDBgui**. Nonostante questo cambio necessario, l'idea di far avviare il programma attraverso un solo tasto è rimasta invariata.

Questa strategia ha permesso di compilare il codice, tuttavia durante i test ci siamo imbattuti in dei **problemi di compatibilità** tra i diversi computer del team. Testando lo stesso progetto su PC differenti, l'ambiente mostrava un comportamento instabile: un'impostazione funzionante su un dispositivo spesso non lo era su un altro, e viceversa. È emerso che basta una minima variazione di configurazione tra i sistemi, come una leggera differenza nei percorsi delle cartelle o nella gestione dell'ambiente virtuale, per impedire ai programmi di comunicare e di funzionare correttamente.

A questo problema si è aggiunto un ulteriore **limite di portabilità**: WSL è uno strumento sviluppato esclusivamente per i sistemi Windows. Questa caratteristica impedisce agli utenti con dispositivi supportati da un sistema diverso da Windows di accedere al progetto.

Alla luce di queste problematiche, abbiamo deciso di cambiare il supporto usato, continuando tuttavia a seguire la strategia dell'emulazione.

Soluzione finale (tramite Docker):

Per eliminare i problemi di compatibilità e svincolare il progetto dai percorsi del sistema locale, ci siamo rivolti a **Docker**.

Questo strumento ci ha permesso di superare i limiti di WSL, in quanto racchiude al suo interno tutto il necessario (software, librerie e la corretta struttura dei percorsi) per far funzionare il programma. Grazie a Docker, adesso è possibile far avviare il lavoro su PC differenti in maniera identica, senza incappare in errori dovuti alle variazioni dell'ambiente virtuale o del sistema ospite.

Tabella di confronto tra i due approcci:

Parametri	Metodo tramite WSL	Metodo tramite Docker
<u>Setup iniziale</u>	Manuale (richiede installazione di pacchetti e script)	Automatizzato (gestito interamente da Docker Desktop)
<u>Portabilità</u>	Bassa (sensibile a differenze di percorsi e variabili)	Assoluta (ambiente isolato del Docker)
<u>SO richiesto</u>	Windows 10/11 con WSL2 attivo, Ubuntu	Windows 10/11
<u>Software da installare</u>	Toolchain ARM, QEMU, Make	Docker Desktop

Possibilità di una “Dual-Mode”:

Durante la fase centrale della produzione, per preservare il lavoro svolto e tenere traccia dei cambiamenti adottati, avevamo deciso di mantenere all'interno del progetto il supporto tramite WSL, facendo **coesistere** le due differenti modalità d'uso (WSL e Docker).

Nel corso dell'integrazione, all'interno del repository abbiamo notato la presenza di conflitti di permessi amministrativi, dovuta al fatto che entrambi i metodi condividevano la stessa cartella. Per aggirare questo problema, abbiamo creato una cartella di build separata chiamata `build_docker`, progettata per essere usata esclusivamente dai container. In questo modo, la modalità WSL continuava a fare riferimento alla cartella `build` standard, consentendo ai due supporti di coesistere sullo stesso repository senza generare collisioni o interferenze.

Tuttavia, in fase conclusiva, la modalità Dual-Mode è stata **definitivamente scartata** per motivi di efficienza e peso della ZIP. Di conseguenza anche il supporto tramite WSL è stato rimosso dal progetto.

Analisi e studio di fattibilità:

Il progetto è iniziato con l'analisi della repository originale per valutare la possibilità di un **porting nativo**. Di seguito vengono riportate le fasi dello studio e le evidenze riscontrate.

1. Analisi strutturale del repository:

- **Identificazione sorgenti:** È stata analizzata la struttura delle cartelle, individuando i file core nella directory `src`.
- **Scoperta critica:** L'analisi dei file ha rivelato che il cuore del sistema (`main.s`) è scritto in **Assembly ARM nativo**.
- **Lo standard dei file:** Mentre il `Makefile` produce file **ELF**, Windows richiede lo standard **PE**.
- **Ulteriori incongruenze tra i sistemi:** È stata evidenziata la differenza tra i separatori di percorso: Windows utilizza `\`, mentre il `Makefile` utilizza `/`.
- **Conseguenza tecnica:** Il porting "nativo" (compilazione per x86_64 Windows) è stato dichiarato **non fattibile** per incompatibilità di set di istruzioni. La strategia è stata ricalibrata sull'**Emulazione Cross-Platform**.

2. Configurazione ambiente di sviluppo:

- **Ambiente di sviluppo integrato (IDE):** Configurazione di VS Code con estensioni dedicate per PowerShell e C/C++.
- **Terminale:** Shift totale dal terminale CMD/Git Bash a **PowerShell 7**, per sfruttare la gestione degli oggetti e l'integrazione con il sistema operativo.
- **Policy di sicurezza:** Abilitazione delle `Execution Policy` (`RemoteSigned`) per permettere l'esecuzione di script di automazione locali.

3. Sviluppo dello studio di fattibilità (approccio iniziale tramite WSL):

- **Test di compilazione "nudo":** I tentativi di compilazione su Windows tramite GCC sono falliti per la mancanza di librerie POSIX (come `unistd.h`), confermando la necessità di un ambiente Linux.

- **Integrazione iniziale WSL:** Abbiamo implementato **WSL** e l'emulatore **QEMU**, unendo il **debugger GDB** all'interfaccia grafica web **GDBgui** per mostrare i registri della CPU sul browser.
- **Automatizzazione:** Per rendere l'installazione automatica, sono stati implementati due script: `setup_arm.sh` per installare in automatico le dipendenze Linux e creare un ambiente virtuale Python per il debugger, e `run_debug.sh` per coordinare l'avvio di QEMU e GDBgui.
- **Problemi di portabilità:** Eseguendo vari test su PC differenti, abbiamo riscontrato una forte **instabilità del sistema** che ne impediva la corretta riproducibilità: gli script fallivano quando eseguiti su un dispositivo diverso da quello utilizzato per la stesura iniziale. Analizzando il problema, abbiamo attribuito questa criticità alle discrepanze di configurazione tra i vari computer, che causavano conflitti tra i programmi.

4. Abbandono di WSL e approccio tramite Docker:

- **Transizione a Docker e abbandono di WSL:** Per evitare i conflitti di configurazione e svincolare il lavoro dai percorsi dei singoli PC, ci siamo rivolti a **Docker**. Questa scelta ha comportato il totale **abbandono di WSL** e dei relativi script (`setup_arm.sh` e `run_debug.sh`), i quali sono stati sostituiti da un unico file di configurazione chiamato `dockerfiles`. Attraverso l'uso di un container non è più necessario configurare manualmente l'ambiente virtuale.
- **Ottimizzazione Docker:** Per velocizzare i tempi di caricamento, l'ambiente Docker è stato basato sulla distribuzione **Alpine Linux**, inserendo la variabile `WERKZEUG_RUN_MAIN`, in modo tale da stabilizzare il debugger ed evitare possibili crash all'avvio.

5. Valutazione di una Dual-Mode:

- **Implementazione Dual-Mode:** Durante l'introduzione di questa modalità, sono stati riscontrati dei conflitti di permessi sui file condivisi. Per aggirare questo problema è stata creata la cartella `build_docker` dedicata solo ai container.
- **Scarto finale della modalità:** A seguito di una serie di considerazioni sulla pesantezza del progetto, la **modalità Dual-Mode è stata**

definitivamente scartata. Mantenere la possibilità di accedere tramite WSL (opzione non strettamente necessaria e con limitate capacità di porting) avrebbe appesantito il pacchetto con istruzioni e script ormai obsoleti.

6. Criticità della sincronizzazione dei processi:

- **Problemi di sincronia tra GDBgui e QEMU:** A seguito dei test svolti all'interno dei container, abbiamo riscontrato un problema legato alla gestione asincrona dei programmi: l'avvio dello script portava a un **arresto prematuro di QEMU** dopo una singola esecuzione, provocando lo spegnimento automatico del container Docker. QEMU si predispondeva per il debug ma, a causa dei tempi di caricamento del backend Python di **GDBgui**, non riceveva una connessione immediata, portando alla terminazione della sessione. Dato che il ciclo di vita di un container è strettamente legato alla persistenza del suo processo principale, l'arresto dell'emulatore portava anche al fallimento dell'intero ambiente. A seguito dell'interruzione del container era necessario lanciare nuovamente l'intero programma.
- **Ricerca di una soluzione:** Per ovviare a questa criticità, il team si è concentrato inizialmente sulla modifica del **Makefile**. Proseguendo con l'analisi del problema, lo sviluppo ha preso strade diverse e sono state raggiunte **due soluzioni**, entrambe con **risultati soddisfacenti**, che affrontavano lo stesso problema con approcci differenti. Difatti, il progetto è stato diviso in **due versioni** sviluppate separatamente, una per ogni approccio adottato.
- **Soluzione ARM-debugger:** Una delle due soluzioni, per superare la criticità dell'arresto dell'emulatore, modifica l'architettura del progetto **rimuovendo il Makefile**, che precedentemente si occupava della sincronizzazione. Il cambiamento più importante consiste **nell'aggiunta di un ciclo while** all'interno del file **entrypoint**, strutturato per mantenere QEMU costantemente in esecuzione per evitare la sua chiusura e, conseguentemente, quella del container. Solo in seguito a questa stabilizzazione viene avviato il backend di GDBgui, assicurando che l'interfaccia trovi la porta di comunicazione già attiva e pronta a ricevere la connessione.

- **Soluzione Arm-gdk:** Questa seconda versione del progetto adotta un approccio fortemente architetturale, basato sulla scomposizione dell'ambiente in **quattro container indipendenti**, coordinati e sincronizzati da un **orchestrator**. Questa separazione garantisce un controllo migliore sui singoli processi e permette, all'occorrenza, di effettuare manutenzione o modifiche su un singolo container senza intaccare gli altri. Il coordinamento dei vari script di sistema è affidato a **gdb-wrapper**, concepito per agire come intermediario e **impedire problemi di asincronia** tra i programmi, assicurando così il corretto funzionamento di QEMU e GDBgui. Nello specifico, il wrapper apre QEMU in background e parallelamente esegue un ciclo di monitoraggio continuo per verificarne lo stato. Solo dopo aver constatato che l'emulatore è attivo e la porta di comunicazione è pronta a ricevere i file, lo script fa partire il backend di GDBgui.
- **Conflitti dell'interfaccia grafica:** Durante la fase di test abbiamo riscontrato un **potenziale conflitto** tra le due versioni qualora vengano installate ed estratte contemporaneamente. Nello specifico, l'interfaccia della prima variante rischia di **sovrasciversi** e comparire al posto di quella della seconda e una volta manifestatasi, questa anomalia si ripresenta a ogni successivo avvio del secondo progetto, impedendo la corretta visualizzazione della sua interfaccia. Il problema si verifica indipendentemente dall'ordine con cui vengono aperti i progetti. È importante precisare che entrambe le versioni **mantengono inalterate le proprie funzionalità principali**, pertanto, l'errore è puramente di natura grafica. A causa delle tempistiche ristrette e considerando che il normale funzionamento del progetto non viene in alcun modo compromesso, l'origine di questo conflitto non è stata approfondita. L'individuazione e la risoluzione della causa principale sono state pertanto rimandate agli sviluppi futuri del progetto.

Tabella di confronto tra le due versioni:

Parametri	ARM-debugger	Arm-gdk
-----------	--------------	---------

<u>Soluzione all'asincronia</u>	Aggiunto ciclo while nell'entrypoint	Introduzione dello script <code>gdb-wrapper</code>
<u>Makefile</u>	Assente	Presente
<u>Impostazione</u>	Funzionale	Architetturale
<u>Container</u>	Solo immagine	4 container
<u>Scelta immagine</u>	Python Slim-Bullseye (Debian)	Ubuntu (generale), Linux Alpine (per container dedicato a orchestrator)
<u>Ambiente virtuale</u>	Python	Ubuntu

Miglioramenti possibili:

Grazie all'esperienza maturata dal team durante lo sviluppo, abbiamo individuato diverse **migliorie** che porterebbero grandi benefici al progetto, ma che non possono essere implementate entro le tempistiche a nostra disposizione.

Pertanto, abbiamo stilato di seguito una lista dei cambiamenti più significativi che valorizzerebbero il lavoro svolto finora:

- **Miglioramenti funzionali:**

- Aggiunta di flag visibili all'interno del registro CPSR: Al momento **l'interfaccia grafica non permette di mostrare i flag** del registro **CPSR** quando necessario, impedendo così a quest'ultimo di adempiere correttamente alla propria funzione.
- Risoluzione di errori legati ai breakpoint: Durante la fase di test abbiamo riscontrato **due anomalie**. Da un lato, alcuni breakpoint non risultano selezionabili. Dall'altro, tra quelli selezionabili, l'esecuzione di alcuni sembra venire saltata poiché il programma passa direttamente al breakpoint successivo; in realtà, verificando i registri, l'esecuzione avviene correttamente, ma non viene visualizzata a schermo.

- **Miglioramenti estetici:**

- Completa visualizzazione dell'output: Al momento **l'output** è visibile soltanto nella **versione Arm-gdk**. Durante lo sviluppo di **ARM-debugger** è stata data la priorità alla risoluzione della criticità legata a QEMU, classificando la corretta formattazione e visualizzazione dell'output

come un miglioramento estetico e, di conseguenza, a **priorità secondaria**.

- Risoluzione dell'errore di interfaccia: Come accennato nello studio di fattibilità, in determinate circostanze può verificarsi la **sovrapposizione dell'interfaccia grafica** tra le due versioni del progetto. Questa anomalia non si manifesta su tutti i dispositivi, ma rappresenta comunque una criticità potenziale. Al momento, l'unico modo per visualizzare correttamente l'interfaccia della versione compromessa consiste nell'eliminare tutti i dati relativi a Docker e procedere a una loro completa reinstallazione, assicurandosi di avviare per prima la versione desiderata. Tuttavia, anche in seguito a questa procedura, il problema tende a ripresentarsi a parti invertite, ovvero mostrando l'interfaccia dell'altro progetto.
- Aggiungere modalità d'uscita user-friendly: Attualmente, per chiudere l'applicazione è necessario arrestare prima il terminale (con modalità che variano a seconda della versione). Tuttavia, l'importanza di questo passaggio rischia di sfuggire a un utente principiante, con il rischio di generare errori dovuti a una chiusura scorretta. Per rendere il progetto **più accessibile**, una possibile miglioria consisterebbe nell'introdurre un metodo di chiusura più intuitivo che sostituisca l'interazione con il terminale, oppure nel gestire la corretta terminazione del processo tramite la chiusura diretta della scheda o della finestra del browser.
- “Azzerare” i valori dei registri nel pre-runtime: Durante la fase di **pre-runtime**, l'emulatore mostra nei registri **R1** e **SP** i valori effettivi di inizializzazione della memoria. Sebbene questo comportamento sia **corretto** architetturealmente e i **registri funzionino fedelmente** rispetto a un sistema ARM Linux reale, la presenza di valori diversi da zero prima dell'esecuzione del programma potrebbe disorientare l'utente. Il rischio è quello di trasmettere l'errata impressione che la CPU contenga dei residui di sessioni precedenti. Per garantire una **maggiore chiarezza visiva**, sarebbe opportuno introdurre una miglioria puramente estetica che uniformi a 0x0 lo stato iniziale di tutti i registri core all'avvio.